# Ders 13 Genişletilmiş Notu: Seminer Sunumları: NNCP, RLE ve Dinamik Huffman, DCQLAC Ses Sıkıştırma, BWT-MTF-Aritmetik Kodlama, Araç Verisi, XML Sıkıştırma ve IDBE

## Ders 13 Genişletilmiş Notu: Seminer Sunumları: NNCP, RLE ve Dinamik Huffman, DCQLAC Ses Sıkıştırma, BWT-MTF-Aritmetik Kodlama, Araç Verisi, XML Sıkıştırma ve IDBE

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Dersin Yapısı
- 2. Sunum 1: NNCP, Sinir Ağlarıyla Kayıpsız Veri Sıkıştırma
- 3. Sunum 2: Görüntü ve Metin Sıkıştırmada Dinamik Huffman ve RLE
- 4. Ara Konu: Bildiri, Dergi ve Hakem Süreci
- 5. Sunum 3: Dinamik Kümelemeli Nicemleme ile Kayıpsız Ses Sıkıştırma (DCQLAC)
- 6. Sunum 4: Cep Telefonu için BWT, Move-to-Front ve Aritmetik Kodlama
- 7. Sunum 5: Araç Verisi için Maksimum Entropi Sinir Ağı ve Optimize Huffman
- 8. Sunum 6: XML Sıkıştırma Tekniklerinin Karşılaştırılması
- 9. Sunum 7: Akıllı Sözlük Tabanlı Kodlama (IDBE)
- Örnek Sorular


## 1. Dersin Yapısı

Bu ders, önceki dersle süren seminer haftalarının bir parçasıdır. Her sunumda bir öğrenci, veri sıkıştırmayla ilgili yayımlanmış bir makaleyi ekran paylaşarak anlattı. Sunan kişi sayısı fazla olduğu için sunumlar yaklaşık 15 dakikayla sınırlandı. Hoca sunum sırasında anlamadığı yerleri sordu, sonuç tablolarını yorumlattı ve konuyu derste görülen yöntemlere (RLE, Huffman, aritmetik kodlama, BWT, PPM) bağladı.

Yedi sunum yapıldı. İlki ve beşincisi sinir ağlarını sıkıştırmada kullanan çalışmalardı. İkinci sunum RLE ile dinamik Huffman'ı görüntü ve metin üzerinde karşılaştırdı; arkasından aynı yazarların ses çalışmasına da bakıldı. Üçüncü sunum kayıpsız ses sıkıştırma üzerineydi. Dördüncü ve yedinci sunumlarda Burrows-Wheeler dönüşümü (BWT) metin sıkıştırmada ön işlem olarak kullanıldı. Altıncı sunum XML sıkıştırıcılarını karşılaştıran bir derlemeydi. BWT bu derste birkaç sunumda ayrı ayrı anlatıldı; bir öğrenci de bunu sunumunda belirtti.

## 2. Sunum 1: NNCP, Sinir Ağlarıyla Kayıpsız Veri Sıkıştırma

### 2.1 Amaç

İlk sunumda bir öğrenci, kayıpsız verilerde sinir ağlarının kullanılmasını anlatan NNCP çalışmasını sundu. Makalenin sorusu şudur: bir sinir ağı kullanarak kayıpsız veri sıkıştırma yapı-

labilir mi? Dersin başından bilindiği gibi büyük verilerin aktarılması ve saklanması için sıkıştırma gerekir; kayıpsız sıkıştırmada amaç boyutu küçültmek, orijinali birebir geri elde etmek ve bunu hızlı yapmaktır.

> **Ek bilgi (derste anlatılmadı):** NNCP büyük olasılıkla Fabrice Bellard'ın geliştirdiği "Lossless Data Compression with Neural Networks" çalışması ve aynı adlı programdır. Bu tür sıkıştırıcılarda model, önceki sembollere bakarak sonraki sembolün olasılık dağılımını tahmin eder ve bu dağılım bir aritmetik kodlayıcıya verilir.

### 2.2 Çalışma mantığı

Sunan öğrenciye göre ağ, veriyi bloklar halinde işler, bloklardan bir model öğrenir ve sonra sıkıştırılacak metni bu modele göre sıkıştırır. GPT-2 gibi büyük dil modelleriyle de deneme yapılmıştır. Makalede LSTM yapısı ve katmanları ayrıntılı anlatılır; Transformer modeli de kullanılır.

Avantaj olarak, sinir ağı eğitildiği için sıkıştırma oranının diğer yöntemlerden yüksek çıkabildiği söylendi. Model küçük ya da büyük veriyle eğitilebilir; büyük veriyle eğitilen model daha başarılıdır ve daha çok yerde kullanılabilir. Yöntem, sıkıştırma dışında sıralı veri işleme ve zaman serisi analizinde de kullanılabilir.

### 2.3 enwik9 sonuçları

Makaledeki tablo, Wikipedia'dan alınmış 1 GB'lık enwik9 dosyası için şöyledir (ekrandan, 00:10:00):

| Program ya da model | Sıkıştırılmış boyut (bayt) | Oran (bpb) |
|---|---|---|
| gzip -9 | 322 591 995 | 2,58 |
| xz -9 | 197 331 816 | 1,58 |
| CMIX (v17) | 116 394 271 | 0,93 |
| Transformer | 130 440 948 | 1,04 |
| LSTM (large1) | 128 292 351 | 1,03 |
| LSTM (large2) | 125 623 896 | 1,00 |

Buradaki bpb, bayt başına bit demektir: sıkıştırılmış boyutun 8 ile çarpılıp orijinal boyuta bölünmesidir. Örneğin gzip için $322\,591\,995 \cdot 8 / 10^9 \approx 2{,}58$ bulunur (python ile doğrulandı). large1 ve large2, büyük veriyle eğitilmiş farklı büyüklükteki modellerdir; large2 daha büyük bir modeldir.

### 2.4 Hocanın soruları: giriş katmanına ne verilir?

Hoca, giriş katmanına neyin verildiğini sordu. Örneğin 512 baytlık bir blok alınırsa, ağın girişinde 512 nöron mu vardır, yani her karakter bir nörona mı verilir? Sunan öğrenci makalede böyle bir bilginin geçmediğini, her karakterin bir nörona verilmediğini söyledi; makalede blok şeması yoktur, yalnızca matematiksel formüller vardır.

Hoca sıkıştırmada ya sözlük tabanlı bir yapı ya da olasılıklar kullanıldığını hatırlattı. LSTM'de bir önceki kelimelerden sonra hangi kelimenin gelebileceğinin olasılıkları da öğrenilir; sıkıştırmanın arka planda bunu kullanıyor olabileceğini belirtti. Makalenin altı sayfalık olduğunu, yarısının LSTM'i anlattığını, yarısının sonuçları verdiğini ve bu yüzden anlaşılmasının zor olduğunu ekledi.

### 2.5 Programın çalıştırılması

Sunan öğrenci programın kodunu indirip kendi bilgisayarında çalıştırdı. Programın kendi test verisi klasik bir XML dosyasıdır. Hoca burada önemli bir soru sordu: XML ve HTML dosyalarında sürekli tekrar eden kalıplar (etiketler) vardır, bu yüzden sıkıştırılmaları kolaydır. Normal bir metin verilseydi aynı başarı elde edilir miydi?

Öğrenci, Wikipedia'dan aldığı düz bir metinle de denedi. Ekrandaki çıktılar şöyledir (ekrandan, 00:40:00):

```
./nncp c ./test/enwik4 ...   STEP 19  SIZE 10000  CSIZE 7049  BPS 5.639
./nncp c ./test/ytu ...      STEP 10  SIZE 5541   CSIZE 4296  BPS 6.202
```

Çıktının başındaki parametrelerde `model=trf` ve `n_layer=4` görünür; yani çalıştırılan sürüm 4 katmanlı bir Transformer modelidir. XML dosyası 10 000 bayttan 7 049 bayta, düz metin 5 541 bayttan 4 296 bayta indi. Bayt başına bit: $7049 \cdot 8/10000 \approx 5{,}64$ ve $4296 \cdot 8/5541 \approx 6{,}20$ (python ile doğrulandı). Düz metin bayt başına daha çok bit gerektirdi, yani daha az sıkıştı. Hoca bunun üzerine artık rahatça yorum yapılabileceğini, tekrar eden yapıdaki verinin daha iyi sıkıştığını söyledi.

> **Ders sonrası not:** Sunumda "STEP" sütunundaki 19 ve 10 değerleri, verinin geçtiği katman sayısı olarak yorumlandı. Aynı çıktıda katman sayısı ayrıca `n_layer=4` olarak verildiği için bu yorum doğru değildir; "STEP" büyük olasılıkla işlenen adım (blok) sayısıdır ve dosya küçüldükçe azalır.

## 3. Sunum 2: Görüntü ve Metin Sıkıştırmada Dinamik Huffman ve RLE

### 3.1 Problem

İkinci sunumda bir öğrenci, "Image and Text Compression Using Dynamic Huffman and RLE Coding" adlı çalışmayı anlattı (ekrandan, 00:20:00). Problem kablosuz iletimdir: bir sinyali aktarmak yüksek bant genişliği ister, bazı frekanslar az, bazıları çok yoğun kullanılır ve havadan iletilen veri trafiği hızla arttığı için kaynak sorunu doğar. Bu konudaki iki zorluk düşük bant genişliği gerektiren veri ve spektrum algılamadır (kullanılan frekans spektrumunun izlenmesi, boş alanların bulunup kullanıcılarla paylaşılması). Amaç, sinyali daha küçük bant genişliğiyle iletmektir.

### 3.2 Yöntem

Çalışmada kayıpsız iki yöntem, RLE ve dinamik Huffman, metin ve görüntü verisi üzerinde karşılaştırılır. Ölçütler sıkıştırma oranı ve sıkıştırma süresidir.

- Görüntüler: farklı boyuttaki görüntülerin işlenmesi uzun sürdüğü için hepsi gri tonlamaya ve 64x64 boyutuna getirilir.
- Metinler: ön işlem olarak karakterlerin ASCII karşılıkları alınır.

Hoca, renkli bir resmi gri tonlamaya çevirmenin renk bilgisini kaybettirdiğini, yani bunun kendi başına kayıplı bir işlem olduğunu belirtti ve buna neden gerek duyulduğunu sordu. Öğrenci, sebebin işlem süresi olduğunu söyledi. Hoca, 2012 dolaylarına ait bir çalışmada bunun anlaşılabilir olduğunu, ama günümüz için yetersiz kaldığını ekledi.

### 3.3 Görüntü sonuçları

Ekrandaki tablo (ekrandan, 00:26:40):

| Görüntü | Huffman CR (%) | Huffman süre (sn) | RLE CR (%) | RLE süre (sn) | MMSE | PSNR (dB) |
|---|---|---|---|---|---|---|
| Leena.png | 7,727 | 0,596 | 47,406 | 0,206 | 255,00 | 24,06 |
| Cameraman.tif | 1,398 | 0,546 | 47,281 | 0,234 | 236,33 | 23,73 |
| Parrot.jpg | 8,713 | 0,692 | 63,672 | 0,217 | 250,58 | 23,98 |
| Atc_radr.bmp | 17,661 | 0,513 | 44,809 | 0,225 | 249,03 | 23,96 |

Slayttaki sonuç cümlesi: RLE bütün resim örneklerinde dinamik Huffman'a göre daha iyi sıkıştırma oranı verir ve işlemi daha kısa sürede yapar.

Hoca önce oranın nasıl tanımlandığını sordu: "compression ratio" 7,7 ise 100 - 7,7 ile yaklaşık %92'lik bir sıkıştırma mı demektir? Makalede tanım verilmemişti. Gri tonlamalı bir resimde RLE'nin %47 vermesi hocaya az geldi.

### 3.4 Kayıpsız sıkıştırmada MSE neden sıfır değil?

Tabloda MMSE ve PSNR değerleri vardır. Sunan öğrenci, yöntemler kayıpsız olduğu için yöntemden kaynaklı bir hata olmadığını, hatanın iletimde kullanılan kanalın gürültüsünden geldiğini ve MSE ile PSNR ile ölçüldüğünü söyledi. Makalede metinler için böyle bir hata hesaplanmamış, hatta metinde hata olmadığı yazılmıştı.

Hoca şu çıkarımı yaptı: kayıpsız sıkıştırmada MSE zaten sıfır çıkmalıdır. Hata kanaldan geliyorsa, içeriğin görüntü ya da metin olması önemli değildir; kanal için ikisi de bir dosyadır. Bu durumda metinde de gürültü olmalıdır. Ayrıca MSE ve PSNR genelde görüntü için kullanılan ölçütlerdir, çünkü kayıplı görüntüde gözün algılamadığı kayıplar ölçülür.

Hoca bir de, tabloda PSNR'ın tek sütun olduğuna dikkat çekti. İki yöntemin sıkıştırma oranları farklıyken PSNR değerleri ikisi için aynı olamaz; değerin hangi yönteme ait olduğunun makalede yazılması gerekir.

### 3.5 Metin sonuçları

Ekrandaki tablo (ekrandan, 00:30:00):

| Metin | Huffman CR (%) | Huffman süre (sn) | RLE CR (%) | RLE süre (sn) |
|---|---|---|---|---|
| Example1.txt | 59,091 | 0,628 | 44,318 | 0,205 |
| Example2.txt | 58,088 | 0,289 | 47,794 | 0,084 |
| Example3.txt | 81,250 | 0,289 | 47,917 | 0,084 |
| Example4.txt | 45,019 | 0,302 | 43,220 | 0,095 |
| Example5.txt | 42,700 | 0,30 | 44,413 | 0,103 |
| Example6.txt | 42,514 | 0,322 | 46,257 | 0,104 |

Example1-3 kısa, Example4-6 uzun metinlerdir. Kısa metinlerde dinamik Huffman'ın oranı daha yüksektir; metin uzadıkça RLE daha iyi sıkıştırır. RLE her iki durumda da daha hızlıdır.

### 3.6 RLE neden metin uzunluğuna bağlı olsun?

Hoca RLE'nin bu kadar başarılı olmasına şaşırdı ve şunu sordu: RLE arka arkaya tekrar eden şeyleri kodlar. Metin uzun da olsa kısa da olsa, karakter düzeyinde arka arkaya tekrar (Türkçede "saat" kelimesindeki "aa" gibi) aynı oranda görülür; uzunluk bunu değiştirmemelidir. Hocanın önerdiği açıklama: karakterler karakter olarak değil, ASCII karşılıklarının bitleri olarak alınmış ve RLE 1 ve 0'ların arka arkaya gelişine uygulanmış olabilir. O zaman sonuç makul olur.

Aynı yazarların ses çalışmasındaki RLE adımları bu açıklamayı destekler (ekrandan, 00:41:15): ses ikilik bitlere çevrilir, her 8 bitlik veride 1 ve 0'lar sayılır, ardışık 1'lerin ve 0'ların sayısı saklanır.

```
RLE kodlama (ses için, makaledeki adımlar):
Adım 1: Bitleri saklamak için boş dizi oluştur.
Adım 2: Girdi sesini ikilik bitlere çevir.
Adım 3: Her 8 bitlik ikilik veride 1 ve 0'ları say.
Adım 4: Bütün ses için tekrarla.
Adım 5: Çıktı dizisinde sakla.
Adım 6: Ardışık 1 bitlerinin sayısını sakla.
Adım 7: Ardışık 0 bitlerinin sayısını sakla.
Adım 8: Son bite kadar 6 ve 7'yi tekrarla. Çıktı: sayılar ve bitler.
```

### 3.7 Sonuç ve gelecek çalışma

Makalenin sonucuna göre görüntüde RLE hem daha kısa sürede hem daha iyi oranla sıkıştırır; metinde kısa metinlerde dinamik Huffman, uzun metinlerde RLE daha iyidir. Gelecek çalışma olarak kablosuz ortamda kayıplı ve kayıpsız başka tekniklerin denenmesi önerilmiştir.

Hoca, bir çalışmanın “gelecek çalışmalar” kısmının sonradan yapılıp yapılmadığına bakmanın çok önemli olduğunu söyledi: yazarlar bunu iş olsun diye mi yazmış, yoksa gerçekten üzerine gitmişler mi? Sunan öğrenci, aynı yazarların ses verisi üzerine bir çalışma yaptığını buldu.

### 3.8 Seste hangi yöntem başarılı?

Hoca bütün sınıfa sordu: RLE’yi de Huffman’ı da biliyoruz; ses verisinde hangisi daha başarılı olur? Sesten özellik çıkarmak için MFCC (altyazıda “Melkeps”) gibi yöntemler olduğunu da hatırlattı.

Ses çalışmasının tablosu (ekrandan, 00:41:15):

| Ses | Boyut | Huffman CR (%) | Huffman süre (sn) | RLE CR (%) | RLE süre (sn) |
|---|---|---|---|---|---|
| Sound1 | 28 kb | 87,16 | 1,97 | 87,5 | 65,4 |
| Sound2 | 47 kb | 87,44 | 2,45 | 87,5 | 103,4 |
| Sound3 | 1059 kb | 87,46 | 2,69 | 87,5 | 113,6 |
| Sound4 | 1513 kb | 87,44 | 2,61 | 87,5 | 116,1 |

Sıkıştırma oranları neredeyse aynıdır (fark virgülden sonradadır), ama RLE çok daha uzun sürer. Hoca, RLE’den önce başka bir ara işlem (bit düzeyine çevirme ve kod kelimesi atama adımları) yapıldığı için sürenin uzadığını tahmin etti.

> **Hocanın vurgusu:** İki yöntemin sıkıştırma başarısı aynıysa, zaman mı yer tasarrufu mu önemli diye bakılır. İkisi aynıysa zaman öne alınır; bu durumda seste dinamik Huffman kullanılır.

## 4. Ara Konu: Bildiri, Dergi ve Hakem Süreci

İkinci sunumdaki çalışmanın bir sempozyum bildirisi olduğu görülünce hoca araştırma yapanlar için genel bir uyarıda bulundu.

> **Hocanın vurgusu:** Herhangi bir konuda araştırma yaparken, özellikle tezlerde, her zaman dergi makalelerine öncelik verin. Dergi makalesi daha iyi bir değerlendirmeden geçer.

Hocaya göre bu çalışma içerik olarak zayıftı ve sorularına cevap vermiyordu; bir dergide kabul edilmesi mümkün değildi, çünkü bütün bu soruların bir dergi makalesinde yanıtlanmış olması gerekir. Uluslararası her konferansın kalitesi aynı değildir. Kaliteyi belirleyen şeylerden bazıları şunlardır:

- Hakemlere gönderilen metinden yazar bilgisinin çıkarılıp çıkarılmadığı. Hakem yazarı bilirse tanıdığı, beğendiği ya da beğenmediği biri olduğu için farklı bir gözle bakabilir; bu etik değildir.
- Kaç hakeme gönderildiği. Bazı kongreler iki hakeme gönderir; bir ret bir kabul gelirse, kabul edilmesi gereken bildiri sayısı dolmamışsa kongre başkanı bildiriyi kabul edebilir. Bazıları bu duruma düşmemek için üç hakeme gönderir.
- Hakemin kararını gerekçelendirip gerekçelendirmediği. Bazı kongrelerde yalnızca “kabul” ya da “ret” gelir, neden kabul ya da ret edildiği yazılmaz.

## 5. Sunum 3: Dinamik Kümelemeli Nicemleme ile Kayıpsız Ses Sıkıştırma (DCQLAC)

### 5.1 Giriş

Üçüncü sunumda bir öğrenci, dinamik kümelemeli nicemleme ile kayıpsız ses sıkıştırmayı anlattı. Ses sıkıştırmanın amacı sesin boyutunu düşürmektir; kayıplı ve kayıpsız olarak ikiye ayrılır. Kayıpsız sıkıştırma, sesin her parçasının gerekli olduğu savunma, biyomedikal ve adli alanlarda gerekir. Müzik iletiminde ise kayıp çoğu zaman göz ardı edilir; insan bunu algılamaz ve çoğu ses sıkıştırma yöntemi kayıplıdır.

### 5.2 Literatür

Sunumdaki zaman çizelgesi (ekrandan, 00:50:00):

```
2003 MPEG-4 ALS          2004 MPEG-4 ALS (RLS-LMS)   2006 MPEG-4 SLS
2010 CELP                2013 Sparse Linear Predictor 2014 OLS-NLMS
2017 FLAC-MD5            2020 WavPack                 2020 Monkey's Audio
```

Sunan öğrencinin anlattıkları:

- MPEG-4 ALS bit hızını düşüren bir çalışmaydı ve hata artığının kodlanması fikrini getirdi; sıkıştırma başarısı yaklaşık %48 civarındaydı.
- RLS ve LMS'i birleştiren yapı (Recursive Least Squares) performansı ve kod çözme hızını artırdı.
- MPEG-4 SLS yaklaşık %50 başarıya ulaştı, ama aritmetik kodlama kullandığı için şişmelere yol açtı.
- CELP daha hızlı kod çözmeye yönelik bir yaklaşımdır. Sparse Linear Predictor %50'nin üzerinde sıkıştırmayı sıradan bir sürede sağlar.
- OLS ve NLMS, hesaplama karmaşıklığını azaltan filtrelerdir.
- FLAC, MD5 ile bir çalışma yapar; anlık başarısı yaklaşık %70'tir. WavPack %50'nin, Monkey's Audio %60'ın üzerinde başarı elde eder.

Altyazıda bu adlar bozuk geçti ("EPEC4 LS", "SERP", "VowPack" gibi); doğru yazımlar slayttan alındı.

### 5.3 Yöntemin genel yapısı

Yöntem sesi tek bir yönteme bağlı kalmadan birkaç adımda sıkıştırır: örnekleme, kümeleme, her kümede nicemleme ve elde edilen bit akışının BWT ile Huffman'dan geçirilmesi. Kod çözmede tersi uygulanır: önce Huffman ve BWT geri çevrilir, bit akışından nicemleme biti bulunup seviyeler geri kurulur, kümeler birleştirilir ve orijinal sinyal elde edilir. Her küme ayrı ayrı işlenir; kümelerin nicemleme boyutu farklı olabilir.

Hoca BWT'yi bilmeyenler için kısaca açıkladı: BWT'deki T "transform"dur. Amacı sıkıştırmadan önce veriyi hazırlamak, yani daha iyi sıkıştırılabilsin diye veriyi uygun bir forma getirmektir. Bir öğrenci bunun dalgacık (wavelet) dönüşümüyle ilgisi olup olmadığını sordu; sunan öğrenci BWT'nin Burrows-Wheeler dönüşümü olduğunu, adın kişilerden geldiğini, dalgacıkla ilgisi olmadığını söyledi.

### 5.4 Örnekleme ve dinamik kümeleme

Örnekleme aşamasında Nyquist ölçütü kullanılır: sinyaldeki en yüksek frekans $f_{max}$ ise örnekleme frekansı $f_s > 2f_{max}$ seçilir.

Kümelemede sinyalin dinamik aralığı bulunur: maksimum genlik ile minimum genlik arasındaki aralık. Bu aralık küme sayısına bölünerek küme aralıkları belirlenir; örneğin 4 küme isteniyorsa aralık 4 parçaya ayrılır.

### 5.5 Nicemleme

Slayttaki algoritma (ekrandan, 00:56:25):

```
1. Küme 1 ile başla.
2. İlk nicemleme bitini 2 seç; Küme 1'deki örnekleri 2 bitle nicemle.
3. MSQE'yi hesapla ve eşikle karşılaştır.
   Eşikten düşükse 4'e git; değilse MSQE eşiğin altına düşene kadar 2'yi tekrarla.
4. Nicemlenmiş seviyeleri ikili bitlerle kodla.
Kalan kümeler (2, 3, 4) için aynı adımları sırayla uygula.
```

Slayttaki tanımlar: $L$ nicemleme düzeylerinin toplam sayısı, $n$ nicemleme biti, $\Delta$ nicemleme adımı, $U_m$ ilgili aralıktaki maksimum giriş değeridir.

$$L = 2^n, \qquad \Delta = \frac{2U_m}{L}, \qquad \text{eşik} = \frac{\Delta^2}{12}$$

Nicemleme biti 2 olduğunda $2^2 = 4$ seviye vardır; bunlar 00, 01, 10, 11 ile kodlanır. MSQE, nicemlenmiş veri ile orijinal veri arasındaki ortalama kare hatadır. Sonuçta bütün kümelerden 0 ve 1'lerden oluşan bir bit akışı elde edilir.

### 5.6 BWT ve Huffman

Bit akışının sıkıştırılması (ekrandan, 00:58:20):

```
1. Nicemlenmiş sinyalin ikili akışını 5'er bitlik gruplara ayır.
2. İlk 32 gruba BWT uygula; benzer çıktı sembolleri art arda gelir.
3. Adım 2'yi bütün ikili temsil üzerinde tekrarla.
4. Sonuca Huffman uygulayarak sembolleri dönüştür ve sıkıştır.
Stereo seste her iki kanala aynı işlem uygulanır.
```

Slayttaki BWT örneğinde girdi `ABCDDCBADBCA$` dizisidir. `$` dizinin sonunu gösteren işarettir. Önce dizinin bütün döndürmeleri yazılır (ilk karakter sona alınarak), sonra satırlar sıralanır ve son sütun alınır. Sıralamada `$` en küçük karakter kabul edilir. Sıralı döndürmeler ve son sütunları (ekrandan, 00:58:20):

```
$ABCDDCBADBCA  -> A
A$ABCDDCBADBC  -> C
ABCDDCBADBCA$  -> $
ADBCA$ABCDDCB  -> B
BADBCA$ABCDDC  -> C
BCA$ABCDDCBAD  -> D
BCDDCBADBCA$A  -> A
CA$ABCDDCBADB  -> B
CBADBCA$ABCDD  -> D
CDDCBADBCA$AB  -> B
DBCA$ABCDDCBA  -> A
DCBADBCA$ABCD  -> D
DDCBADBCA$ABC  -> C
```

Çıktı `AC$BCDABDBADC` olur (python ile doğrulandı). Sıralama aynı karakterleri yan yana getirdiği için sonraki sıkıştırma kolaylaşır; örneğin yan yana üç E gelirse "3E" denebilir. Dönüşüm tersine çevrilebilir, yani aynı çıktıdan orijinal geri üretilebilir.

### 5.7 Kod çözme

Slayttaki adımlar (ekrandan, 01:00:00):

```
Adım 1 (ters Huffman ve BWT):
  1. Kökten başla, bir yaprak bulunana kadar devam et.
  2. Geçerli bit 0 ise ağacın sağ düğümüne git.
  3. Bit 1 ise sol düğüme geç.
  4. Yaprağa ulaşınca o karakteri yaz ve 1. adımdan devam et.
  5. Ters BWT ile çözülmüş bit akışını yeniden oluştur.
Adım 2 (dekuantizasyon):
  1. Blok yapısından nicemleme bitini bul.
  2. Bit akışını nicemlenmiş değerlere çevir
     (nicemleme biti 2 ve akış 0001100011 ise 2'şer bitlik gruplara böl).
  3-4. Bunu her kümenin bütün bit akışı için tekrarla.
Adım 3: Kümeleri genlik aralıklarına göre sırayla birleştir.
```

### 5.8 Sonuçlar

Deneylerde beş şarkı kategorisi (Rabindra Tagore, klasik, rock, pop, sufi) kullanıldı ve üç kayıpsız teknik (Monkey's Audio, WavPack, FLAC) rakip olarak alındı. Sıkıştırma oranları (ekrandan, 01:01:45):

| Kategori | Monkey’s Audio ort. (%) | WavPack ort. (%) | FLAC ort. (%) | DCQLAC ort. (%) |
|---|---|---|---|---|
| Rabindra Tagore | 54,08 | 53,69 | 72,68 | 95,51 |
| Klasik | 62,99 | 53,33 | 68,25 | 95,69 |
| Rock | 62,44 | 53,90 | 69,24 | 94,76 |
| Pop | 57,81 | 53,80 | 72,01 | 95,29 |
| Sufi | 51,85 | 54,75 | 74,33 | 95,32 |
| Genel ortalama | 57,83 | 53,89 | 71,30 | 95,31 |

Önerilen yöntem bütün kategorilerde rakiplerini geçti. Hoca bu oranın çok yüksek olduğunu belirtti; sunan öğrenci de ilk gördüğünde %95’i ütopik bulduğunu söyledi.

Altyazıdan aktarılan diğer ortalamalar (ekranda doğrulanmadı):

- Kodlama hızı: Monkey’s Audio 0,17, WavPack 0,15, FLAC 0,12, önerilen 0,10.
- Kod çözme hızı: Monkey’s Audio 0,18, WavPack 0,15, FLAC 0,13, önerilen 0,11.
- Entropi: Monkey’s Audio 13,06, WavPack 13,07, FLAC 13,05, önerilen 13,81.
- SNR: Monkey’s Audio 31, WavPack 32, FLAC 31, önerilen 34,03. PSNR’da önerilen yöntem yaklaşık 55 ile öndedir.

  **Ders sonrası not:** Sunumda SNR’ın yüksek olmasının kötü olduğu söylendi. Doğrusu SNR’da da PSNR’da olduğu gibi yüksek değer, gürültünün sinyale göre daha az olduğu, yani daha iyi bir sonuç demektir. Bu tabloya göre önerilen yöntem SNR’da da öndedir.

Çalışma 2020 tarihlidir. Hoca yöntemin ticari bir ürüne dönüşüp dönüşmediğini sordu; bu bilinmiyordu.

### 5.9 Soru-cevap

**Soru:** Bir yerde nicemleme biti 2 bit olarak söylendi, ama ses 16 bit kullanıyor. 2 bit kayıplı sıkıştırma yapmaz mı? 16 bit daha mantıklı görünüyor. **Cevap:** Sunan öğrenciye göre yöntem değerleri 2 bitlik parçalara bölerek ifade ediyor. Normalde 8 bitlik bir değer 16 parçaya bölünebilir ya da bit hızı değiştirilebilir; burada parçaların parçaları kodlandığı için 2 bit yetiyor. Aşamalar örnekleme, kümeleme ve nicemleme sırasıyla ilerliyor.

## 6. Sunum 4: Cep Telefonu için BWT, Move-to-Front ve Aritmetik Kodlama

### 6.1 Amaç

Dördüncü sunum, “Text Data Compression for Mobile Phone Using Burrows-Wheeler Transform, Move-to-Front Code and Arithmetic Coding” çalışmasıydı. Cep telefonlarının belleği sabit ve küçüktür; bellek kullanımını optimize etmek ve veri iletimini hızlandırmak için metin sıkıştırma gerekir. Çalışmada cep telefonundaki Kur’an ve hadis uygulamaları ele alınmış, referans olarak bir hadis uygulaması kullanılmıştır.

Sunan öğrenci veri sıkıştırmayı şöyle özetledi (ekrandan, 01:10:00): verileri daha kompakt hale getirmek; veri ses, metin, resim ya da video olabilir; kayıpsız sıkıştırmada orijinal veri geri elde edilir ve çoğunlukla metinde kullanılır (RLE, Huffman, LZW, aritmetik kodlama); kayıplı sıkıştırmada veri orijinalden farklı olabilir ve multimedyada kullanılır (dönüşüm kodlama, dalgacık); kayıplının sıkıştırma oranı daha yüksektir.

Çalışmada dört düzen karşılaştırılır:

- Şema 1: yalnız aritmetik kodlama.
- Şema 2: Move-to-Front + aritmetik kodlama.
- Şema 3: BWT + aritmetik kodlama.
- Şema 4: BWT + Move-to-Front + aritmetik kodlama.

### 6.2 Aritmetik kodlama

Aritmetik kodlama, her giriş sembolünü belirli bir bit koduyla değiştirme fikrini atlar; girdi dosyasındaki sembol dizisini tek bir kayan noktalı sayıyla değiştirir. Bu sayı $0 \leq x < 1$ aralığındadır ve sembol akışını tek biçimde geri vermek üzere çözülebilir. Mesaj uzadıkça bu sayı için daha çok bit gerekir.

Slayttaki örnek (ekrandan, 01:11:40): "FATIMAH AS" metninde 10 karakter vardır. Her karakterin olasılığı sayısı bölü 10'dur; A üç kez geçtiği için 3/10, diğerleri 1/10'dur. Aralıklar olasılıklarla orantılı verilir:

| Karakter | Olasılık | Aralık |
|---|---|---|
| boşluk | 1/10 | $0{,}00 \leq r < 0{,}10$ |
| A | 3/10 | $0{,}10 \leq r < 0{,}40$ |
| F | 1/10 | $0{,}40 \leq r < 0{,}50$ |
| H | 1/10 | $0{,}50 \leq r < 0{,}60$ |
| I | 1/10 | $0{,}60 \leq r < 0{,}70$ |
| M | 1/10 | $0{,}70 \leq r < 0{,}80$ |
| S | 1/10 | $0{,}80 \leq r < 0{,}90$ |
| T | 1/10 | $0{,}90 \leq r < 1{,}00$ |

### 6.3 Move-to-Front

Move-to-Front (MTF) veriyi kendisi sıkıştırmaz, ama fazlalığı azaltmaya yardım eder. Her sembol, "son kullanılan semboller" listesindeki indeksiyle değiştirilir; kullanılan sembol listenin başına alınır. Böylece aynı sembolün uzun tekrarları 0'a dönüşür, uzun süredir görülmeyen semboller büyük sayılar alır.

Slayttaki örnek (ekrandan, 01:12:50): dizi "aaafhhimst", başlangıç listesi alfabe sırasıdır.

```
a: liste abcdefg...   indeks 0  -> liste değişmez
a: indeks 0
a: indeks 0
f: indeks 5           -> liste fabcdeghij...
h: indeks 7           -> liste hfabcdegij...
h: indeks 0
i: indeks 8           -> liste ihfabcdegjk...
m: indeks 12          -> liste mihfabcdegjkl...
s: indeks 18          -> liste smihfabcdegjkl...
t: indeks 19          -> liste tsmihfabcdegjk...
Çıktı: 0, 0, 0, 5, 7, 0, 8, 12, 18, 19
```

Sonuç python ile doğrulandı.

### 6.4 Burrows-Wheeler dönüşümü

BWT, MTF ve aritmetik kodlamadan önce kullanılır ve veri bloğunu yeniden düzenler; bir veri dönüşüm algoritmasıdır. Slayttaki örnekte (ekrandan, 01:14:15) "fathimahas" dizisinin ilk karakteri her adımda sona kaydırılır; satırlar son karakterlerine göre alfabetik sıralanır ve çıktı "aaafhhimst" olur. Sıralamanın amacı çıktıda tekrar eden karakterlerin art arda gelmesi, böylece sıkıştırmanın kolaylaşmasıdır.

> **Ders sonrası not:** Standart BWT'de döndürmeler baştan (ilk karakterden itibaren) sıralanır ve son sütun alınır. "fathimahas" için standart BWT çıktısı `mhfsathiaa` olur (python ile). Makaledeki tabloda çıktı yalnızca harflerin alfabetik sıralanmış hali olduğundan, ek bilgi olmadan orijinal metne geri çevrilemez; tablo büyük olasılıkla hatalı ya da basitleştirilmiştir. Bir önceki sunumdaki `ABCDDCBADBCA$` örneği ve son sunumdaki "bahamalar" örneği standart tanıma uyar.

### 6.5 Deney ve sonuçlar

Bu makalede sıkıştırma oranı şöyle tanımlanır:

$$\mathrm{CR} = \frac{\text{sıkıştırılmış dosya boyutu}}{\text{orijinal dosya boyutu}} \times 100$$

En iyi algoritma en küçük oranı verendir. Deney verisi olarak 32 bölümlük bir hadis kitabının metin dosyaları kullanıldı; bölüm boyutları yaklaşık 5 000 bayttan 801 000 bayta kadar değiştiği için dosya boyutu çeşitliliği sağlar. Her şemada girdi dosyası kodlayıcıdan geçip çıktı dosyası olur, çıktı dosyası da kod çözücüden geçip orijinal dosyayı verir.

- Şema 1 (yalnız aritmetik): en iyi oran %58, en kötü %75. İlk bölümde yaklaşık 5 900 baytlık dosya yaklaşık 4 400 bayta indi.
- Şema 2 (MTF + aritmetik): en iyi %36, en kötü %54. İlk bölümün sıkıştırılmış boyutu 4 496 bayttan 3 213 bayta indi.
- Şema 3 (BWT + aritmetik): 1024, 2048, 4096 ve 8192 baytlık bloklarla denendi; en iyi %59, en kötü %76. Aritmetik kodlamanın tek başına sonucuna benzer.
- Şema 4 (BWT + MTF + aritmetik): en iyi sonuç, yaklaşık %36,2.

Daha büyük BWT blok boyutu daha iyi sıkıştırma verir; 8192 baytlık blok en iyisidir. Sunan öğrencinin açıklamasına göre büyük blok, MTF ile tam sayı koduna çevrilecek daha çok karakter demektir. Çalışma, sonucun diğer hadis uygulamalarında da kullanılabileceğini ve daha küçük oran için daha büyük BWT bloğu denenebileceğini önerir.

## 7. Sunum 5: Araç Verisi için Maksimum Entropi Sinir Ağı ve Optimize Huffman

### 7.1 Problem

Beşinci sunum, “Vehicle Text Data Compression and Transmission Methods Based on Maximum Entropy Neural Networks and Optimized Huffman Encoding Algorithms” çalışmasıydı. Servis ve toplu taşıma araçlarında, araç multimedya cihazına benzeyen araç terminalleri vardır. Bunlar konum, hız, yükseklik, plaka, araç durumu, sürüş ve yolcu koşulları gibi bilgileri toplar ve kablosuz bağlantıyla kurumun sunucusuna gönderir. Amaç bu verileri daha iyi sıkıştırmak ve gönderim süresini azaltmaktır (ekrandan, 01:30:00).

Huffman ya da LZW gibi algoritmalar veriyi sıkıştırabilir, ama şehirdeki iletişim ortamının farklılıklarını ve düşük güçlü iletimi hesaba katmaz. Her verinin ayrı ayrı sıkıştırılması eş zamanlı gönderimi geciktirir. Terminal donanımı bir bilgisayar ya da telefon kadar güçlü olmadığından yöntem az bellek kullanmalı ve hafif olmalıdır.

Sinir ağları önerilir, ama büyük modeller terminalde çalışmaz; terminali buluttaki bir modele bağlamak da gizlilik riski doğurur. Çözüm terminalde daha küçük bir model kullanmaktır. Model üç adımda küçültülür: önemsiz bağlantılar kesilir, ağırlıklar nicemlenir ve Huffman kodlaması uygulanır.

### 7.2 Maksimum entropi modeli

Sinir ağıyla sıkıştırma yüksek oran ve açma hızı sağlar, ama verinin iki kez taranması gerekir: önce model eğitilir, sonra veri modele göre kodlanıp gönderilir. Yazarlar, örnek verilerden gerçek olasılık dağılımıyla tutarlı bir istatistiksel model kurmak ister ve bunun için maksimum entropi ilkesini seçer. Bir $g(x)$ algoritmasıyla, $P(y \mid x)$ olasılıkları tutarlı olana kadar $w$ ağırlıkları ayarlanır. Bu model hem koşullu kısıtları hem de kısıtlar altındaki maksimum entropiyi sağlayan olasılıksal bir modeldir.

Bu istatistiksel model bir sinir ağına çok benzer çalıştığı için sinir ağı olarak kurulmuştur. Bağlama dayalı karakter olasılık dağılımını tahmin etmek için iki katmanlı bir ağ kullanılır: her olası bağlam bir giriş nöronu $x_i$, her olası çıkış karakteri bir çıkış nöronu $y_j$ ile gösterilir ve her giriş-çıkış çifti bir $w_{ij}$ ağırlığıyla bağlanır. Sunan öğrenci bunu iki katmanlı bir yoğun (dense) katman gibi düşünmeyi önerdi. $y_i$ çıkışı, sonraki karakterin $i$ olma olasılığıdır.

### 7.3 Optimize Huffman

Huffman, gönderilecek GPS ve metin verisinin kayıpsız sıkıştırması için kullanılır. Konum bilgisi gibi sürekli tekrar eden gereksiz veriler silinir ve bir ara bellekte saklanır. Geleneksel Huffman büyük ara bellek ve yüksek karmaşıklık ister; bu sorunları çözmek için Huffman ağacı yedi adımla geliştirilir. İlk iki adım (ekrandan, 01:40:00):

```
(1) İkili Huffman ağacı başlatılır; keyfi bir kök düğüm seçilir ve ağırlığı 0 yapılır.
(2) Kodlanmayan yeni karakterler için yeni düğümdeki iki düğüm üst düğümün
    ağırlığını birleştirir; ağırlığı 0 olan diğer düğüm yeni bir 0 ağırlığı tanımlar.
```

Sunan öğrencinin aktardığı etkiler: kök düğüm ağırlığının 0 yapılmasıyla sembol akışının taranma sayısı bire iner; sinir ağı ağırlıkları ayarlandıktan sonra hesaplamada yalnız bir olasılığı olan karakter taranır; yalnızca düğüm numaraları değiş tokuş edilir. Bu hem ara bellek kullanımını hem algoritmanın karmaşıklığını azaltır.

**Soru:** Kök düğümün ağırlığını sıfıra ayarlamak taranması gereken sayıyı nasıl bire indirir? **Cevap:** Sunan öğrenci, normal Huffman’da önce bütün verinin taranıp frekansların çıkarıldığını, sonra ağaç kurulurken verinin bir kez daha tarandığını hatırlattı. Burada önceden maksimum entropi için ayarlanmış iki katmanlı ağ olasılıkları verdiği için verinin bir kez taranmasının yettiğini anladığını söyledi.

Hoca, ağdaki ağırlıkların arka arkaya gelen iki karakterin olasılığı olarak kullanılıp kullanılmadığını ve neden bu olasılıkların doğrudan kullanılmadığını sordu. Sunan öğrenci makalede yalnızca bu yedi adımın sözel olarak verildiğini, örnek olmadığını söyledi. Hoca, “optimize Huffman” adından Huffman ağacının daha hızlı kurulduğunu çıkarabildiğini, ama ara adımları makaleden tam çıkaramadığını belirtti; 2019 tarihli bir çalışma olduğu için “optimized Huffman encoding” araması yapılırsa nasıl gerçekleştirildiğinin bulunabileceğini ekledi. Optimizasyonun dinamik ağacın gönderilmesi, kodlar arasındaki varyansın düşürülmesi, beklemenin azaltılması ve daha hızlı kodlama ile çözme gibi noktaları kapsadığını söyledi.

### 7.4 Deney ve sonuçlar

Önce kırsal bir ortamda internet hızına bakıldı: araç terminalinden sunucuya 240 ICMP paketi gönderildi, ping süreleri ve karşıya ulaşan paket sayısı ölçüldü. Ping, sunucuya veri gönderip onay alma arasındaki gecikmedir, milisaniyeyle ölçülür ve düşük olması iyidir. Paket kaybı ortalama yaklaşık %4-4,5 çıktı.

Sonra veri dört biçimde gönderildi: sıkıştırmadan (orijinal), yalnız Huffman ile, yalnız LZW ile ve önerilen yöntemle. Veri özelleştirilmiş bir toplu taşıma aracından 300 dakika boyunca toplandı; veri toplama çevrimi 15, 10 ve 5 saniye olarak ayarlandı. Sonuç tablosu (ekrandan, 01:44:10):

| Algoritma | Çevrim (sn) | Sıkıştırılmış boyut (kb) | Sıkıştırma oranı | Aktarım süresi (sn) | İletim hata oranı | Paket kaybı |
|---|---|---|---|---|---|---|
| Orijinal | 15 | 824,56 | %0,00 | 72,66 | %3,36 | %5,52 |
| Orijinal | 10 | 1 478,83 | %0,00 | 142,35 | %3,84 | %5,94 |
| Orijinal | 5 | 2 578,57 | %0,00 | 265,39 | %3,59 | %5,04 |
| Huffman | 15 | 423,22 | %48,67 | 33,37 | %2,87 | %4,54 |
| Huffman | 10 | 752,48 | %49,12 | 75,54 | %2,79 | %4,78 |
| Huffman | 5 | 1 314,65 | %49,02 | 127,88 | %2,98 | %5,17 |
| LZW | 15 | 371,38 | %54,96 | 29,78 | %2,67 | %4,54 |
| LZW | 10 | 653,11 | %55,84 | 67,76 | %2,61 | %4,61 |
| LZW | 5 | 1 156,83 | %55,14 | 112,39 | %2,80 | %4,32 |
| Maks. entropi NN + Huffman | 15 | 336,43 | %59,20 | 27,76 | %2,42 | %3,35 |
| Maks. entropi NN + Huffman | 10 | 613,43 | %58,52 | 61,35 | %2,32 | %3,42 |

| Algoritma | Çevrim (sn) | Sıkıştırılmış boyut (kb) | Sıkıştırma oranı | Aktarım süresi (sn) | İletim hata oranı | Paket kaybı |
|---|---|---|---|---|---|---|
| Maks. entropi NN + Huffman | 5 | 1 043,41 | %59,54 | 103,41 | %2,34 | %3,65 |

Makalede sıkıştırma oranının tanımı yazılmamış. Tablodan bunun kazanç oranı olduğu çıkar: 15 saniyelik çevrimde $1 - 336{,}43/824{,}56 \approx 0{,}592$, yani %59,2; Huffman için $1 - 423{,}22/824{,}56 \approx \%48{,}7$ (python ile doğrulandı). Hoca da bu oranın 100'den çıkarılmadan, doğrudan kazanç olarak okunması gerektiğini söyledi.

İletim hata oranı, sunucunun aldığı ve çözdüğü metin verisinin uzunluğunun sıkıştırmadan önceki uzunluğa oranıyla ilgilidir; paket kaybı oranı karşıya hiç ulaşamayan paketlerin miktarıdır. En iyi sıkıştırma ve en kısa aktarım süresi önerilen yöntemdedir.

### 7.5 Hız verisiyle doğruluk testi

Yöntemin gerçek verideki doğruluğunu görmek için bir yolcu otobüsünden 4 gün boyunca 12 600 hız verisi toplandı. Bunların 239'u yanlış yere gitti ya da kayboldu; $12\,600 - 239 = 12\,361$ veri sıkıştırılıp gönderildi. Orijinal hız verileriyle sıkıştırılıp gönderilip açılan hız verileri karşılaştırıldı ve fark ortalama kare hatayla ölçüldü: 0,23. Yazarlar bunu makul bulur; hız verisi %98,1 oranında doğru iletilmiştir. Yöntem aktarım süresini azalttığı için kablosuz iletimin verimliliğini artırır ve farklı baz istasyonları olan şehir bölgelerinde daha verimli olur.

Hoca sunumun sonunda, sonuçların doğrulanamadığını ima ederek yazarlar öyle diyorsa öyle kabul edeceklerini esprili bir dille söyledi.

## 8. Sunum 6: XML Sıkıştırma Tekniklerinin Karşılaştırılması

### 8.1 XML ve sıkıştırma ihtiyacı

Altıncı sunumda bir öğrenci, XML sıkıştırma tekniklerini inceleyip karşılaştıran bir makaleyi anlattı. XML 1998'de sistemler arası veri aktarımı, kayıt tutma ve sorgulanabilir veri saklama için ortaya çıkmış bir formattır. Makale 2008'de yazılmıştır ve XML'i fiili standart olarak anar; günümüzde JSON ve YAML gibi formatlar öne çıkmıştır, ama XML hâlâ SVG gibi formatlarda sık görülür.

XML kendini açıklayıcı (self-describing) bir formattır, çünkü şema her kayıtta tekrarlanır (ekrandan, 01:50:00). Esnek şeması nedeniyle yarı yapılandırılmış bir veri yapısıdır. Şemanın her kayıtta tekrarlanması dosyayı büyütür ve sıkıştırmaya çok elverişli hale getirir. Slayttaki örnek:

```
<Books>
  <Book ISBN="0553212419">
    <title>Sherlock Holmes: Complete Novels...</title>
    <author>Sir Arthur Conan Doyle</author>
  </Book>
  <Book ISBN="0743273567">
    <title>The Great Gatsby</title>
    <author>F. Scott Fitzgerald</author>
  </Book>
  ...
</Books>
```

Makale XML sıkıştırmanın durumunu derlemek, karşılaştırma için bir derlem (corpus) oluşturmak ve araçların performansını ayrıntılı sonuçlarla vermek ister. Sonuçlar bir web sitesinde de yayımlanmıştı, ama site artık erişilebilir değildir.

### 8.2 Sınıflandırma

İki sınıflandırma vardır:

- Sıkıştırılan verinin XML olduğunu anlayıp anlamamasına göre:
  - Genel metin sıkıştırıcılar.
  - XML'in farkında olan sıkıştırıcılar: şemaya ihtiyaç duyan ve duymayan.
- Sorgu desteğine göre:
  - Sorgulanamayanlar.
  - Sorgulanabilenler: homomorfik ve homomorfik olmayan.

### 8.3 Genel metin sıkıştırıcılar

XML metin tabanlı olduğu için genel metin sıkıştırıcılar kullanılabilir. Makalede üç tanesi alınmıştır:

- **gzip:** LZ77 ve Huffman'ı kullanan iki aşamalı deflate algoritması. İlk aşamada LZ77 ile tekrarlanan diziler azaltılır, ikinci aşamada Huffman ile sık sembollere kısa kodlar verilir.
- **bzip2:** katmanlı bir sıkıştırıcıdır. Sıra: ham veriye RLE, sonra Burrows-Wheeler dönüşümü, sonra MTF, tekrar RLE ve en sonda Huffman. Sunan öğrenci BWT'nin tersinin nasıl alındığını özetledi: çıktı bir sütun olarak yazılır ve sıralanır; çıktı bu sütunun başına eklenip yeniden sıralanır; bu işlem dizi uzunluğu kadar tekrarlanınca orijinal döndürmeler, dolayısıyla sıkıştırılmamış metin elde edilir.
- **PPM:** n. dereceden Markov modelleriyle sonraki karakteri tahmin eden istatistiksel bir yöntemdir. Basit olmasına ve bu makaledeki sıkıştırıcılar arasında en verimlisi olmasına rağmen hesaplama açısından çok masraflıdır.

  **Ders sonrası not:** Sunumda PPM "Partial String Matching" diye açıldı; doğru açılımı "Prediction by Partial Matching"tir.

### 8.4 XML'in farkında olan, sorgulanamayan sıkıştırıcılar

İlk örnek XMill'dir (altyazıda "XMIL"). 1999'da AT&T Bell Laboratuvarlarında geliştirilmiştir ve sonraki neredeyse bütün XML sıkıştırıcılarına temel olmuştur. XMill, XML'in etiketlerini ve verisini ayrı ayrı sıkıştırır. Şemaya ihtiyaç duymayan, sözlük tabanlı bir yöntemdir; arka uç olarak gzip, bzip2 ya da PPM gibi metin tabanlı bir sıkıştırıcı seçilebilir.

XMill'i temel alan küçük iyileştirmeler de vardır: biri veriden önce bir okuma yapıp tekrar edenleri sözlükteki daha kısa sembollerle değiştirir, XComp ise bellek kullanımındaki optimizasyonuyla ayrılır. Şemaya ihtiyaç duyan yöntemlerde şema önceden verilir ve önceden bir sözlük hazırlanır; bu daha verimli sıkıştırma sağlar.

Sunan öğrenci, verinin türüne göre kaplara (container) ayrıldığı bir örnek de gösterdi: örneğin bir web günlüğünde sunucu alanından gelenler bir IP kabına, istek satırından gelenler başka bir kaba atılır; bu kapların içinde run-length gibi hazır fonksiyonlar çalıştırılabilir ve sonunda gzip gibi bir metin sıkıştırıcı uygulanır.

Daha yeni bazı araçlar (altyazıda "EXALT" ve "XCHAP") testlerde çok hata almış; biri yaklaşık %96 oranında sıkıştırmayı tamamlayamamıştır. Metin tabanlı yöntemler ise hiç hata almamıştır, çünkü verinin XML ya da düz metin olması onlar için önemli değildir.

### 8.5 Sorgulanabilen sıkıştırıcılar

Bu araçlarda sıkıştırılmış veri üzerinde, açmadan sorgu çalıştırılabilir.

**XGrind** XMill gibi öncü bir çalışmadır; sonraki birçok çalışma onu temel almıştır. Huffman kullanır ve şemaya ihtiyaç duyar. Şema verilince etiketler numaralandırılır, veri Huffman ile sıkıştırılır. Slayttaki örnek (ekrandan, 02:00:00):

```
XML:
<Student SID="S1234">
  <LastName>Smith</LastName>
  <FirstName>Joe</FirstName>
  <MInitial>M</MInitial>
  <Address>620 12th Ave, Coralville</Address>
</Student>
```

```
Dönüşüm sonucu:
T0 A0 nahuff (S1234)
T1 nahuff (Smith) /
T2 nahuff (Joe) /
T3 nahuff (M) /
T4 nahuff (620 12th Ave, Coralville) /
/
```

Her etikete T0, T1, T2, T3, T4 gibi basit bir sembol, niteliğe A0 verilir. `nahuff` başlığı, o alanın Huffman ile sıkıştırılmadığını gösterir; arkasından veri gelir. XGrind homomorfiktir: "homomorfik" orijinaliyle aynı biçimde olmak demektir; sıkıştırılmış veri orijinal XML'in yapısını değiştirmez.

**XPress** de homomorfiktir; Huffman yerine aritmetik kodlama kullanır.

**Homomorfik olmayan** araçlar sorgu performansını artırmak için örneğin ayrı bir indeks yapısı ekler. Bunların en verimlisi XQzip'tir. XQzip verinin yapısını değiştirir: çok iç içe bir XML'de ortak düğümleri bulup birleştirir ve bir bağlı liste ekleyerek ağacı optimize eder; her etiketin içindeki veriye gzip benzeri sıkıştırma uygular. Orijinal yapı değiştiği için homomorfik değildir.

### 8.6 Veri seti ve sonuçlar

Makale çok çeşitli bir veri seti kullanmıştır (ekrandan, 02:02:00): EXI, XMark, XBench, Wikipedia, DBLP, US House, SwissProt, NASA, Shakespeare, Lineitem, Mondial, BaseBall, Treebank ve rastgele üretilmiş dosyalar. Her dosya için boyut, farklı etiket sayısı, düğüm sayısı, maksimum derinlik ve "data ratio" verilir. Data ratio, etiketler atıldığında kalan verinin oranıdır. Java ile rastgele üretilen dosyalarda etiketler açılıp kapanır ama içine bir şey yazılmaz; bu yüzden oranları 0'dır. Wikipedia dosyalarında oran 0,91-0,98'dir; yani bunlar neredeyse tamamen veridir, yapı azdır.

Sonuçlarda sıkıştırma oranı, sıkıştırılmış boyutun sıkıştırılmamış boyuta bölünmesidir. Bu oran, sıkıştırma süresi ve açma süresiyle üç sıralama fonksiyonu kurulmuştur; fonksiyonlar yalnız katsayılarda ayrılır ve katsayı değişimi sıralamayı değiştirmemiştir. En iyi genel performansı, verinin XML olmasıyla ilgilenmeyen gzip vermiştir; onu XMill'in gzip arka uçlu sürümü ve PPM izler.

**Soru:** Yani en başarılı gzip mi? **Cevap:** Evet, ama gzip sıkıştırılmış veri üzerinde sorguya izin vermez. Örneğin bellekte sıkıştırılmış veri tutup açmadan sorgulamak istenirse gzip işe yaramaz. Sorgulanabilen araçlar bu karşılaştırmada test edilmemiş, yalnızca tanıtılmıştır.

## 9. Sunum 7: Akıllı Sözlük Tabanlı Kodlama (IDBE)

### 9.1 Konu

Son sunum, internet üzerinden yüksek hızlı veri iletimi için metin sıkıştırmaya yönelik akıllı sözlük tabanlı kodlama (Intelligent Dictionary Based Encoding, IDBE) çalışmasıydı. Makale 2006 tarihlidir. Makaleye göre bilinen sıkıştırma algoritmaları teorik en iyi sıkıştırma oranına ulaşamaz; alternatif olarak metnin önce dönüştürülmesi, sonra dönüştürülmüş metnin sıkıştırılması önerilir.

### 9.2 Burrows-Wheeler dönüşümü

BWT veriyi kendisi sıkıştırmaz; sıkıştırma öncesi bir ön hazırlık olarak veriyi sıralar. Arkasından MTF, RLE ve aritmetik kodlama gibi aşamalar gelir; kod çözmede tersi uygulanır. Slayttaki örnek (ekrandan, 02:10:00):

```
Girdi kelime: bahamalar

1. Bütün döndürmeler:  2. Alfabetik sıralı:  3. Son sütun:
bahamalar              ahamalarb             b
rbahamala              alarbaham             m
arbahamal              amalarbah             h
```

```
larbahama               arbahamal               l
alarbaham               bahamalar               r
malarbaha               hamalarba               a
amalarbah               larbahama               a
hamalarba               malarbaha               a
ahamalarb               rbahamala               a

Çıktı: bmhlraaaa
```

Çıktıda dört “a” yan yana gelmiştir (python ile doğrulandı). Dönüşüm tersine çevrilebilir: sıralama tekrarlanarak orijinal kelime geri elde edilir.

### 9.3 Yıldız kodlaması

Makalede ilgili çalışma olarak yıldız kodlaması (*-encoding) anlatılır. Elde bir sözlük vardır; sözlükteki her kelime, olabildiğince çok harfi * olan bir kodla gösterilir. Yıldız ne kadar çok olursa sıkıştırma o kadar iyi olur. Slayttaki örnek, Romeo ve Juliet’ten bir metindir (ekrandan, 02:08:35):

```
But soft, what light through yonder window breaks?
It is the East, and Iuliet is the Sunne,
...
Yıldız kodlayıcıdan sonra:
B** *of*, **a* **g** *****g* ***d*r ***do* b*e***?
It *s *** E**t, **d ***i** *s *** *u**e,
```

Hoca, hangi kelimeye kaç yıldız verileceğini kimin belirlediğini sordu (örneğin “soft” kodunda iki yıldız varken başka bir kelimede daha çok yıldız vardır). Sunan öğrenci, makalede yıldız kodlamasının algoritmasının ve sözlüğün nasıl kurulduğunun anlatılmadığını, hazır bir sözlük kullanıldığını söyledi.

### 9.4 IDBE

IDBE’de de bir sözlük oluşturulur ve girilen metin bu sözlüğe göre kodlanır. Sunan öğrenci fikri şöyle açıkladı: insanlar harfleri yanlış yazılmış bir kelimeyi de okuyup anlayabilir; burada amaç kelimeyi bir sözlükte tutup makinenin anlayabileceği kısa bir koda çevirmektir.

- Sözlük oluşturma: metin okunur, kelimeler tabloya eklenir (tabloda yoksa eklenir, varsa eklenmez); kodlar için belirli bir ASCII aralığı kullanılır.
- Kodlama: her kelime alınır; sözlükte varsa, yıldız kodlamasındakine benzer biçimde kısa bir koda çevrilir.
- Örnek metin İncil’den alınmıştır; kodlanmış metin anlamsız karakterlerden oluşur, ama sonradan daha iyi sıkıştırılır.

Makalede kodlama ve sözlük oluşturma algoritmaları verilmiş, ama kod çözmenin nasıl yapılacağı ayrıca anlatılmamıştır; sunan öğrenciye göre makale bunu BWT’nin tersine bağlamaktadır.

### 9.5 Sonuçlar

Makale yalnız BWT, BWT + yıldız kodlaması ve BWT + IDBE’yi karşılaştırır. Calgary derlemi sonuçlarından bazı satırlar (ekrandan, 02:13:25):

| Dosya | Boyut (KB) | BWT BPC | BWT süre (sn) | *Encode BPC | *Encode süre (sn) | IDBE BPC | IDBE süre (sn) |
|---|---|---|---|---|---|---|---|
| bib | 108,7 | 2,11 | 1 | 1,93 | 6 | 1,69 | 4 |
| book1 | 750,8 | 2,85 | 11 | 2,74 | 18 | 2,36 | 11 |
| book2 | 596,5 | 2,43 | 9 | 2,33 | 14 | 2,02 | 10 |
| geo | 100,0 | 4,84 | 2 | 4,84 | 6 | 5,18 | 5 |
| news | 368,3 | 2,83 | 6 | 2,65 | 10 | 2,37 | 7 |
| trans | 91,5 | 1,63 | 2 | 1,53 | 5 | 1,46 | 4 |

IDBE çoğu dosyada en düşük bit/karakter değerini verir; ikili bir veri olan geo dosyasında ise BWT'den kötüdür (ekrandan). Süresi yalnız BWT'den biraz uzun, yıldız kodlamasından kısadır. Aynı karşılaştırma Canterbury derleminden metinlerle de yapılmış ve IDBE yine daha iyi sonuç vermiştir. Sonuç olarak dönüşüm sunucu ve istemci tarafında ek yük getirir, ama veri daha iyi sıkıştırılır.

> **Ders sonrası not:** Sunumda değerler "bit per second" diye okundu; tablodaki birim BPC, yani karakter başına bittir.

Hoca çalışmayı kendi yılları için iyi bir çalışma olarak değerlendirdi; daha yeni bir çalışma olsaydı daha farklı bir değerlendirme olacağını belirtti.

## Örnek Sorular

**Soru 1** (derste sözlü, 00:15:34): XML ve HTML dosyalarında sürekli tekrar eden kalıplar olduğu için sıkıştırılmaları kolaydır. Sinir ağı tabanlı sıkıştırıcıya normal bir metin verilseydi aynı başarı alınır mıydı?

> **Çözüm:** Derste program iki dosyayla çalıştırıldı (ekrandan, 00:40:00). XML dosyası 10 000 bayttan 7 049 bayta indi: $7049 \cdot 8/10000 \approx 5{,}64$ bit/bayt. Düz metin 5 541 bayttan 4 296 bayta indi: $4296 \cdot 8/5541 \approx 6{,}20$ bit/bayt. Düz metin bayt başına daha çok bit gerektirdi; tekrarlı yapıdaki XML daha iyi sıkıştı.

**Soru 2** (derste sözlü, 00:22:05): Renkli bir resim sıkıştırmadan önce gri tonlamaya çevrilirse yapılan sıkıştırma hâlâ kayıpsız sayılır mı?

> **Çözüm:** Hayır. Gri tonlamaya çevirmek renk bilgisini kaybettirir; bu kendi başına kayıplı bir işlemdir. Sonradan kullanılan RLE ya da Huffman kayıpsız olsa da, orijinal renkli görüntü geri elde edilemez.

**Soru 3** (derste sözlü, 00:24:07): Kayıpsız sıkıştırma yapılan bir sistemde görüntüler için sıfırdan farklı MSE ve PSNR raporlanmış, metinlerde ise hata olmadığı söylenmiştir. Bu tutarlı mıdır?

> **Çözüm:** Kayıpsız sıkıştırmada yöntemden kaynaklı MSE sıfırdır. Sıfırdan farklı hata iletim kanalının gürültüsünden geliyorsa, kanal için görüntü ile metin arasında fark yoktur; ikisi de bir dosyadır. Bu yüzden aynı kanaldan gönderilen metinde de hata beklenir; makalenin açıklaması tutarsızdır.

**Soru 4** (derste sözlü, 00:31:13): RLE ardışık tekrarları kodlar. Metin uzadıkça RLE'nin sıkıştırma oranının artması nasıl açıklanabilir?

> **Çözüm:** Karakter düzeyinde ardışık tekrar oranı metnin uzunluğuna bağlı değildir. Hocanın açıklaması, RLE'nin karakterlere değil, ASCII karşılıklarının bitlerine, yani ardışık 1 ve 0'lara uygulanmış olabileceğidir. Aynı yazarların ses çalışmasındaki RLE adımları gerçekten ardışık 1 ve 0 bitlerinin sayılmasıdır (ekrandan, 00:41:15).

**Soru 5** (derste sözlü, 00:37:47; tablo ekrandan, 00:41:15): Ses verisinde RLE mi dinamik Huffman mı daha başarılı olur? Aşağıdaki sonuçlara göre hangisi seçilmelidir?

| Ses | Huffman CR (%) | Huffman süre (sn) | RLE CR (%) | RLE süre (sn) |
|---|---|---|---|---|
| Sound1 | 87,16 | 1,97 | 87,5 | 65,4 |
| Sound2 | 87,44 | 2,45 | 87,5 | 103,4 |
| Sound3 | 87,46 | 2,69 | 87,5 | 113,6 |
| Sound4 | 87,44 | 2,61 | 87,5 | 116,1 |

> **Çözüm:** Sıkıştırma oranları neredeyse aynıdır. Oranlar aynıysa zaman öne çıkar; Huffman 2-2,7 saniyede, RLE 65-116 saniyede sıkıştırdığı için dinamik Huffman seçilir.

**Soru 6** (ekrandan, 02:10:00): "bahamalar" kelimesinin Burrows-Wheeler dönüşümünü bulunuz.

**Çözüm:** Bütün döndürmeler yazılır ve alfabetik sıralanır: ahamalarb, alarbaham, amalarbah, arbahamal, bahamalar, hamalarba, larbahama, malarbaha, rbahamala. Son sütun alınır: `bmhlraaaa` (python ile doğrulandı).

**Soru 7** (ekrandan, 01:12:50): "aaafhhimst" dizisini, başlangıç listesi alfabe sırası olan Move-to-Front ile kodlayınız.

**Çözüm:** a, a, a için indeks 0; f için 5 (f başa alınır); h için 7; ikinci h için 0; i için 8; m için 12; s için 18; t için 19. Çıktı: 0, 0, 0, 5, 7, 0, 8, 12, 18, 19 (python ile doğrulandı).

**Soru 8** (ekrandan, 01:11:40): "FATIMAH AS" metni için aritmetik kodlamada kullanılacak olasılıkları ve aralıkları belirleyiniz.

**Çözüm:** Metinde 10 karakter vardır. A üç kez geçer: 3/10; boşluk, F, H, I, M, S, T birer kez: 1/10. Aralıklar sırayla: boşluk [0,00; 0,10), A [0,10; 0,40), F [0,40; 0,50), H [0,50; 0,60), I [0,60; 0,70), M [0,70; 0,80), S [0,80; 0,90), T [0,90; 1,00).

**Soru 9** (ekrandan, 01:44:10): Araç verisi tablosunda 15 saniyelik çevrimde orijinal veri 824,56 kb, önerilen yöntemle sıkıştırılmış veri 336,43 kb'tır. Tablodaki %59,20 sıkıştırma oranı nasıl tanımlanmıştır?

**Çözüm (derste verilmedi):** $1 - 336{,}43/824{,}56 \approx 0{,}592$, yani %59,2. Oran, sıkıştırmayla kazanılan yerin orijinal boyuta oranıdır; yüksek değer daha iyidir. Aynı tanım Huffman ($1 - 423{,}22/824{,}56 \approx \%48{,}7$) ve LZW ($1 - 371{,}38/824{,}56 \approx \%55{,}0$) için de tutar (python ile doğrulandı).

**Soru 10** (derste sözlü, 01:05:05): DCQLAC'ta ses 16 bitlik olduğu halde nicemleme biti 2 seçiliyor. 2 bitlik nicemleme kayba yol açmaz mı?

**Çözüm:** Derste verilen cevap: yöntem değerleri kümelere, kümeleri de parçalara böldüğü için her parça 2 bitle ifade edilebiliyor; MSQE eşiğin altına inmezse nicemleme adımı tekrarlanıyor.

## Dersin Toparlaması

Bu derste yedi seminer sunumu, dönem boyunca görülen yöntemlerin gerçek çalışmalarda nasıl birleştirildiğini gösterdi. Sinir ağı tabanlı sıkıştırıcılar (NNCP ve araç verisi çalışması) olasılık tahminini bir modele bırakır, ama sonunda yine aritmetik kodlama ya da Huffman gibi klasik bir entropi kodlayıcısına dayanır. RLE ile Huffman karşılaştırmaları, aynı yöntemin veri türüne, uzunluğa ve uygulama biçimine (karakter ya da bit düzeyi) göre çok farklı sonuç verebileceğini gösterdi. BWT birçok sunumda ortak bir ön işlem olarak çıktı: veriyi kendisi sıkıştırmaz, aynı karakterleri yan yana getirerek MTF, RLE ve entropi kodlayıcılarının işini kolaylaştırır; blok büyüdükçe kazanç artar.

Hocanın sorularından çıkan genel ders, sonuç tablolarını eleştirel okumaktır: sıkıştırma oranının nasıl tanımlandığına, kayıpsız sıkıştırmada hata ölçütlerinin anlamına, oranlar eşitken sürenin belirleyici olduğuna ve çalışmanın nerede yayımlandığına bakmak gerekir. Hoca ayrıca araştırmada dergi makalelerine öncelik verilmesini ve "gelecek çalışmalar" kısmının sonradan izlenip izlenmediğinin kontrol edilmesini önerdi.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.